

Assignment 2: Software Design Patterns in Java

1. Identification

- Assignment Title: Assignment 2 — Creational Design Patterns (Factory Method & Abstract Factory)
- Student Name: Sanzhar Abayev
- Group: SE-2538
- GitHub Repository Link: <https://github.com/Sanzh-123/SDP-assignment-2>
- Submitted Commit Hash: 35d61e3

2. Introduction

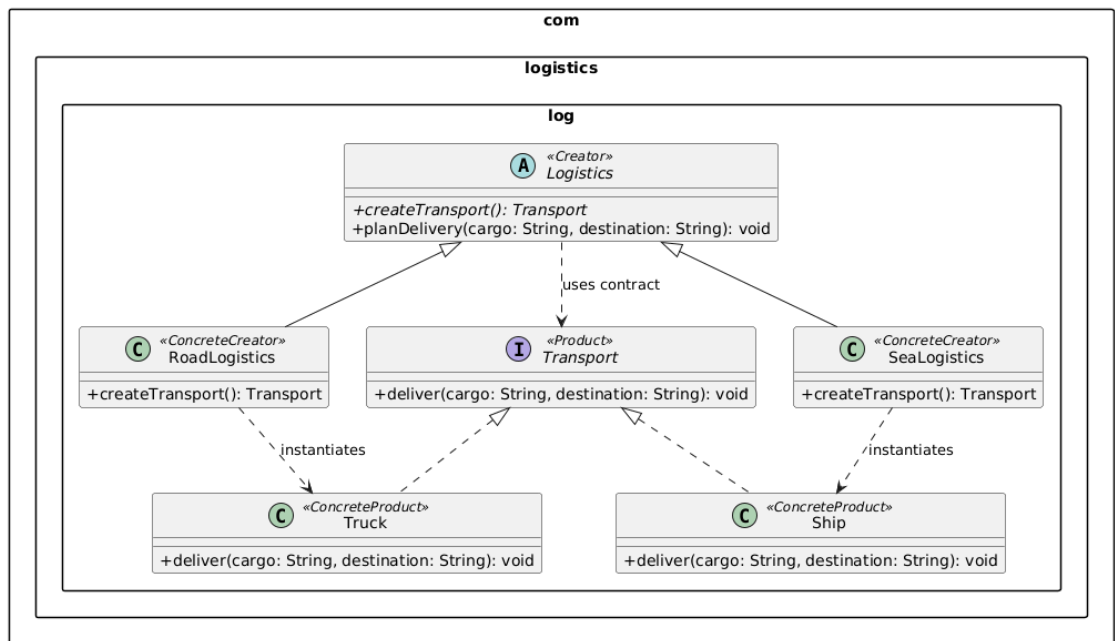
This application is a cross-platform logistics management system built in Java 17. It integrates two creational design patterns—Factory Method and Abstract Factory—to manage transport logistics workflows and platform-specific UI component rendering simultaneously.

- Factory Method Role: Applied in the delivery subsystem (`com.logistics.log`). It handles single product creation where the exact concrete transport type (Truck or Ship) depends on the concrete logistics creator (RoadLogistics or SeaLogistics). This decouples core delivery orchestration from concrete transport classes.
- Abstract Factory Role: Applied in the UI subsystem (`com.logistics.ui`). It creates entire families of related UI controls (Button and Checkbox) corresponding to target platforms (Windows or macOS). This guarantees that UI controls instantiated at runtime match without hardcoding concrete constructors into client code.

3. Structural Architecture & UML Class Diagrams

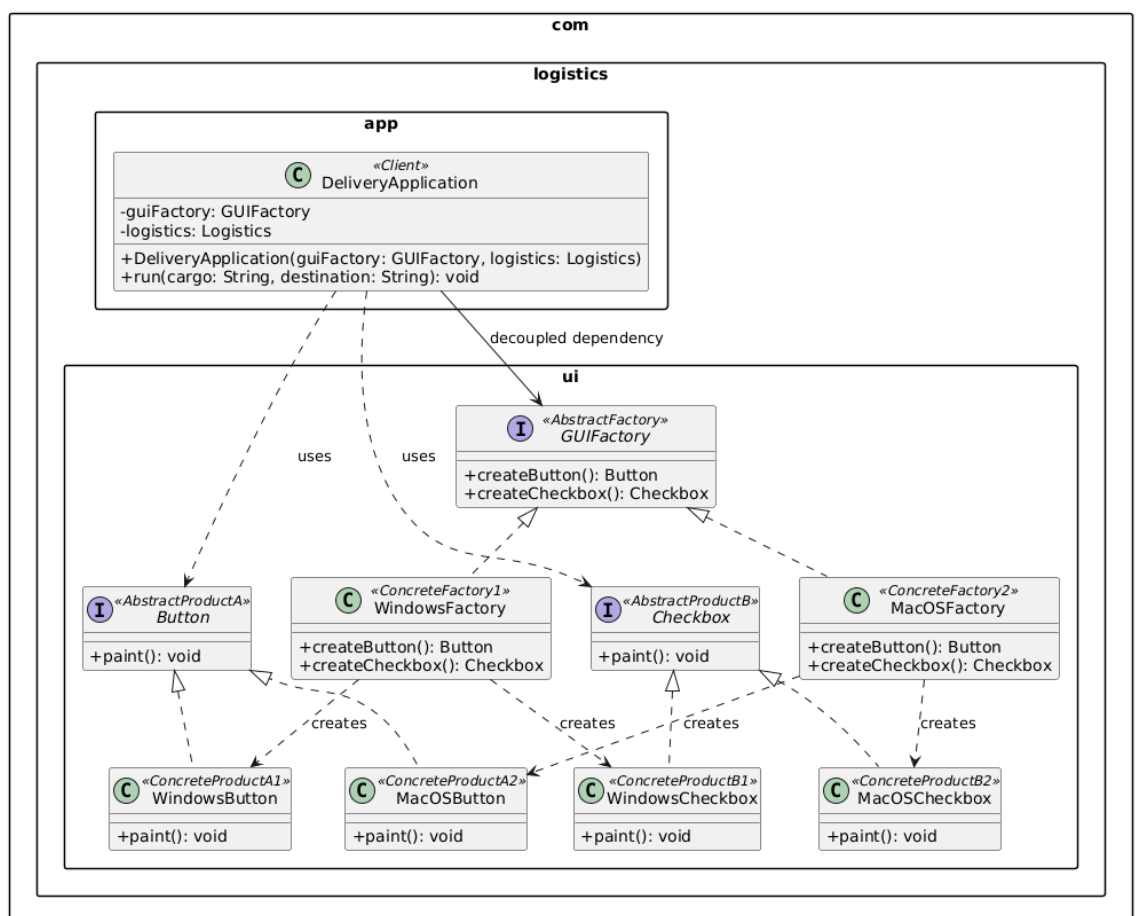
3.1 Factory Method Pattern

The Factory Method pattern defines an abstract creator `Logistics` declaring `createTransport()`, deferring instantiation to `RoadLogistics` and `SeaLogistics`.



3.2 Abstract Factory Pattern

The Abstract Factory pattern provides the **GUIFactory** interface declaring creation methods for **Button** and **Checkbox**. Concrete factories (**WindowsFactory** and **MacOSFactory**) guarantee platform consistency.



4. Clean Code Evidence (Section 7)

1. Meaningful Names

- Excerpt:

```
public interface GUIFactory { ... }
```

```
public class WindowsFactory implements GUIFactory { ... }
```

- Explanation & Benefit: Names like GUIFactory and WindowsFactory clearly state their domain roles, making code self-documenting and eliminating ambiguity between creators, products, and clients.

2. Small Methods

- Excerpt:

```
private static Logistics selectLogistics(String mode) {
```

```
    return switch (mode) {
```

```
        case "ROAD" -> new RoadLogistics();
```

```
        case "SEA" -> new SeaLogistics();
```

```
        default -> null;
```

```
    };
```

```
}
```

- Explanation & Benefit: Parsing and object selection are modularized into single-responsibility static methods containing under 10 lines of code.

3. Avoid Duplicated Logic

- Excerpt:

```
public void planDelivery(String cargo, String destination) {
```

```
    Transport transport = createTransport();
```

```
    transport.deliver(cargo, destination);
```

```
}
```

- Explanation & Benefit: The execution workflow is declared once in the abstract Logistics base class. Concrete creators avoid duplicating the operational sequence, overriding only object instantiation.

4. Data Abstraction (Clean Code Chapter 6)

- Excerpt:

```
public class DeliveryApplication {
    private final GUIFactory guiFactory;
    private final Logistics logistics;
    ...
}
```

- Explanation & Benefit: DeliveryApplication references abstract interface contracts (GUIFactory, Logistics) rather than concrete implementations. State is kept private, concealing internal creation details from callers.

5. Objects and Encapsulation (Clean Code Chapter 6)

- Excerpt:

```
Button button = guiFactory.createButton();
button.paint();
```

- Explanation & Benefit: Object behavior (paint()) is expressed through method calls on abstract interfaces rather than exposing internal variables or requiring accessor methods (getters/setters).

5. Required Verification Results (Section 6)

Check	Input CLI Arguments	Expected Result	Actual Result / Output	Status
1	ROAD WINDOWS	Truck delivery; Windows button & checkbox.	Renders Windows button & checkbox. Truck delivers equipment.	PASS
2	SEA WINDOWS	Ship delivery; Windows button & checkbox.	Renders Windows button & checkbox. Ship delivers equipment.	PASS

Check	Input CLI Arguments	Expected Result	Actual Result / Output	Status
3	ROAD MACOS	Truck delivery; macOS button & checkbox.	Renders macOS button & checkbox. Truck delivers equipment.	PASS
4	SEA MACOS	Ship delivery; macOS button & checkbox.	Renders macOS button & checkbox. Ship delivers equipment.	PASS
5	AIR WINDOWS	Clear validation error message.	Error: Unsupported delivery mode 'AIR'. Expected ROAD or SEA.	PASS
6	ROAD LINUX	Clear validation error message.	Error: Unsupported UI platform 'LINUX'. Expected WINDOWS or MACOS.	PASS
Missing	<i>(No parameters)</i>	Parameter usage warning.	Error: Missing required configuration parameters.	PASS

6. Conclusion & Design Reflection

Pattern Comparison

- Factory Method focuses on a single product hierarchy, relying on inheritance to defer instantiation to creator subclasses (RoadLogistics, SeaLogistics).
- Abstract Factory focuses on product families (Button + Checkbox), using composition to provide an interface that constructs related products without coupling the client to concrete classes.

Extensibility Reflection

- Adding a new transport (e.g., AirLogistics / Plane): Create Plane implements Transport and AirLogistics extends Logistics. Add a branch in Main.java. DeliveryApplication requires no changes.
- Adding a new UI family (e.g., LinuxFactory): Implement LinuxButton, LinuxCheckbox, and LinuxFactory implements GUIFactory. DeliveryApplication requires no changes.
- Adding a new UI component type (e.g., TextField): Modify GUIFactory to declare createTextField(), update all concrete factories (WindowsFactory, MacOSFactory), implement concrete products, and update DeliveryApplication to render the new control.

7. References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
2. Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall (Chapter 6: Objects and Data Structures).